

GeyzsoN / prosperity_rust_backtester

<> Code

Issues

Pull requests

Agents

Actions

Projects

Security and quality

Ins

Watch 0

☆

Unknown, MIT licenses found

Contributing

☆ 65 stars

🔗 32 forks

👁 0 watching

🔗 Branches

📈 Activity

🏷 Tags

🌐 Public repository

🔗

🔗 1 Branch

🏷 5 Tags

🔗

🏷

🔍 Go to file

T

Go to file

Add file ➕

Code

...

 **GeyzsoN**

Release v0.3.0

89d6337 · 15 hours ago

🕒

📁 datasets	Add round 1 data and limits to def...	2 days ago
📁 scripts	Harden local macOS builds and a...	last month
📁 src	Release v0.3.0	15 hours ago
📁 traders	Add round 1 data and limits to def...	2 days ago
📄 .dockerignore	Initial commit	last month
📄 .gitignore	Auto-generate submission JSON ...	3 weeks ago
📄 8191.json	Harden local macOS builds and a...	last month
📄 CHANGELOG.md	Release v0.3.0	15 hours ago
📄 CONTRIBUTING.md	Initial commit	last month
📄 Cargo.lock	Release v0.3.0	15 hours ago
📄 Cargo.toml	Release v0.3.0	15 hours ago
📄 Dockerfile	Harden local macOS builds and a...	last month
📄 LICENSE-APACHE	Initial commit	last month
📄 LICENSE-MIT	Initial commit	last month
📄 Makefile	Add carry mode for continuous m...	3 weeks ago
📄 README.md	Add round 1 data and limits to def...	2 days ago
📄 pyproject.toml	Release v0.3.0	15 hours ago

Rust Backtester

This repo is a self-contained Rust backtester for IMC Prosperity 4.

It only supports local backtesting. There is no API surface and no hosted workflow in this repo.

Everything needed for the default backtest flow now lives inside this directory. The bundled default trader is:

- `traders/latest_trader.py`

Setup

Clone the repo:

```
git clone https://github.com/GeyzsoN/prosperity_rust_backtester.git
cd prosperity_rust_backtester
```



macOS

Install the toolchain once:

```
xcode-select --install
curl https://sh.rustup.rs -sSf | sh
source "$HOME/.cargo/env"
python3 --version
```



Then either install the CLI:

```
make install
```



or just run the backtester directly:

```
make backtest
```



The macOS `make` targets intentionally build through a wrapper instead of your full shell environment. By default they write Rust build artifacts to:

```
~/Library/Caches/rust_backtester/target
```



If you want a different target dir, override it explicitly:

```
CARGO_TARGET_DIR=/path/to/target make build-release
```



Windows

Use WSL2. Open an Ubuntu shell inside WSL2 and run the same commands there. Native Windows shells are not the target environment for this repo.

There is no separate manual build step required for normal use:

- `make backtest` and the other `make run` targets use `cargo run`, which builds automatically on first use
- `make install` installs the CLI once so you can run `rust_backtester` directly afterward
- `make doctor` prints local diagnostics for macOS build hangs and execution-policy issues

Included Data

The repo is organized by round:

- `datasets/tutorial/prices_round_0_day_-2.csv`
- `datasets/tutorial/trades_round_0_day_-2.csv`
- `datasets/tutorial/prices_round_0_day_-1.csv`
- `datasets/tutorial/trades_round_0_day_-1.csv`
- `datasets/tutorial/submission.log`
- `datasets/round1/prices_round_1_day_-2.csv`
- `datasets/round1/trades_round_1_day_-2.csv`
- `datasets/round1/prices_round_1_day_-1.csv`
- `datasets/round1/trades_round_1_day_-1.csv`
- `datasets/round1/prices_round_1_day_0.csv`
- `datasets/round1/trades_round_1_day_0.csv`
- `datasets/round2/`
- `datasets/round3/`
- `datasets/round4/`
- `datasets/round5/`
- `datasets/round6/`
- `datasets/round7/`
- `datasets/round8/`

Right now the bundled public data is the raw IMC tutorial day data in `datasets/tutorial/`, the raw round 1 day data in `datasets/round1/`, plus a sample tutorial `submission.log` produced with the bundled basic trader. The other round folders are there so future round files can be placed in the correct folder instead of being mixed together. If you place a portal `submission.log` file into a round folder, the backtester will use it. `submission.log` is also generated for persisted runs.

CLI

The CLI is intentionally simple:

```
rust_backtester
```



With no arguments it will:

- auto-pick the newest Python file that looks like a trader from this repo's local `scripts/` , `traders/submissions/` , or `traders/`
- default the dataset to the latest populated round folder under `datasets/`
- run in fast mode
- print one compact row per day

That means the simplest local commands are:

```
make backtest
```



or:

```
rust_backtester
```



Round-specific shortcuts:

```
make tutorial
```



Submission and future-round shortcuts are also available once you place a `submission.log` , `submission.json` , or round data into `datasets/round1/` , `datasets/round2/` , and so on.

Useful optional variables for any `make backtest` target:

```
make tutorial DAY=-1
make submission ROUND=round1
make round3 TRADER=traders/latest_trader.py
make round2 PERSIST=1
make tutorial FLAT=1
make tutorial CARRY=1
```



Supported input formats:

- normalized dataset JSON files
- IMC day data as matching `prices_*.csv` and `trades_*.csv`
- portal submission logs such as `submission.log`

Day selection behavior:

- `DAY=-1` or `DAY=-2` runs only that day file inside the round
- `DAY=all` or omitting `DAY` runs the whole round bundle, including any submission file when present
- `make submission ROUND=round1` runs the submission dataset for `datasets/round1/` when a submission file is present

Explicit examples:

```
rust_backtester \
  --trader /path/to/trader.py \
  --dataset tutorial
```



```
rust_backtester \
  --trader /path/to/trader.py \
  --dataset datasets/tutorial
```



```
rust_backtester \
  --trader /path/to/trader.py \
  --dataset /path/to/submission.log
```



```
rust_backtester \
  --trader /path/to/trader.py \
  --dataset datasets/round1
```



Behavior:

- fast mode is the default
- the CLI prints one result row per day
- `--dataset` accepts either a path or a short alias
- when `--dataset` points to a directory, every supported dataset in that directory is run
- `prices_*.csv` files are paired automatically with matching `trades_*.csv` files from the same folder
- `latest` and `tutorial` run the full bundled tutorial round bundle: day `-2`, day `-1`, and the sample tutorial submission log
- use `--day <n>` to run only the matching day dataset within the round bundle; this excludes submission files
- `metrics.json` is always written under `runs/<backtest-id>/`
- default fast runs also write `submission.log` under `runs/<backtest-id>/`
- use `--artifact-mode` to choose which extra artifacts are written
- use `--carry` or `CARRY=1` to carry positions, own trades, market trades, and trader state across non-submission day datasets in the same round
- use `--flat` or `FLAT=1` to place multi-run outputs in a single directory with dataset/day-prefixed filenames
- use `--persist` or `PERSIST=1` to write the full replay artifact set under `runs/`

- persisted multi-day or multi-file runs also write one combined bundle at `runs/<backtest-id>/` , including `combined.log` and `manifest.json`
- for multi-run visualizer uploads, use each child `RUN_DIR/submission.log` ; the top-level bundle does not emit a stitched replay file
- product output defaults to a compact summary so large product sets do not flood the terminal

Bundled dataset aliases:

- `latest`
- `tutorial` , `tut` , `tutorial-round` , `tut-round`
- `round1` , `r1`
- `round2` , `r2`
- `round3` , `r3`
- `round4` , `r4`
- `round5` , `r5`
- `round6` , `r6`
- `round7` , `r7`
- `round8` , `r8`
- `tutorial-1` , `tut-1` , `tut-d-1`
- `tutorial-2` , `tut-2` , `tut-d-2`

Product display modes:

- `--products summary` default: print a separate product table with the top product PnL contributors and an `OTHER(+N)` rollup when needed
- `--products full` : print a separate product table with every product
- `--products off` : show only the per-day total

Artifact modes:

- `--artifact-mode none` : write only `metrics.json`
- `--artifact-mode submission` default when `--persist` is not set: write `metrics.json` and `submission.log`
- `--artifact-mode diagnostic` : write `metrics.json` and `bundle.json` with the PnL series included for diagnostics
- `--artifact-mode full` : write the full persisted artifact set: `metrics.json` , `bundle.json` , `submission.log` , `activity.csv` , `pnl_by_product.csv` , `combined.log` , and `trades.csv`
- `--persist` implies `--artifact-mode full` unless you explicitly override `--artifact-mode`

Flat layout behavior:

- `--flat` only changes multi-run layouts; single-run outputs stay unchanged
- multi-run outputs are written into `runs/<backtest-id>/` with prefixed filenames such as `tutorial-day-2-submission.log` and `tutorial-day-2-metrics.json`
- when `--flat` is combined with `--persist` , the same directory also includes `combined.log` and `manifest.json`

Carry mode behavior:

- `--carry` only applies to non-submission day datasets; submission datasets remain separate runs
- with `--carry` , consecutive day datasets in the same round are merged into one connected replay ordered by `(day, timestamp)`
- positions, prior own trades, prior market trades, and trader data are carried across those merged day boundaries
- carry mode also normalizes timestamps into one continuous timeline, so the first tick of the next day starts immediately after the previous day ends
- a carried run reports `DAY=all` in the summary because it spans multiple days

Artifact mode examples:

```
rust_backtester \  
  --trader /path/to/trader.py \  
  --dataset tutorial \  
  --artifact-mode none
```



```
rust_backtester \  
  --trader /path/to/trader.py \  
  --dataset tutorial \  
  --artifact-mode diagnostic
```



```
rust_backtester \  
  --trader /path/to/trader.py \  
  --dataset tutorial \  
  --artifact-mode full
```



Example output shape:

```
trader: latest_trader.py [auto]  
dataset: tutorial [default]  
mode: fast  
artifacts: log-only  
SET          DAY    TICKS  OWN_TRADES  FINAL_PNL  RUN_DIR  
D-2          -2     10000      39         118.10  runs/backtest-123-day-2-day-  
2  
D-1          -1     10000      42         123.45  runs/backtest-123-day-1-day-  
1  
SUB          -1      2000      18          51.20  runs/backtest-123-  
submission-day-1  
  
PRODUCT      D-2      D-1      SUB  
TOM          70.00    77.20    29.10  
EMR          48.10    46.25    22.10
```



Bundled Targets

```
make doctor
make build
make build-release
make test
make install
make install-pip
make install-uv
make install-uv-editable
make backtest
make tutorial
```



macOS Troubleshooting

If a local Rust build hangs on macOS, the most likely symptom is that `cargo build`, `make build-release`, or `make backtest` stalls during `build-script-build` while `sypolicyd` uses a lot of CPU.

First retry path:

```
make doctor
make build-release
make backtest
```



Those `make` targets already use the repo wrapper and a stable target dir outside the repo.

If it still hangs and you do not want to reboot, the non-restart remediation is:

```
sudo killall sypolicyd
make build-release
```



If local macOS execution policy is still unhealthy, use the isolated fallback:

```
make docker-build
make docker-smoke
```



This is a local macOS executable-launch issue, not a backtester logic issue.

Additional `round1` to `round8` and `round1-submission` to `round8-submission` targets are included in the `Makefile`. They become usable once those dataset folders contain JSON files.

Isolated Verification

There is a Docker-based smoke path for isolated verification:

```
make docker-smoke
```



The Docker image builds the project in a clean container and runs the zero-argument backtest flow during image build.

Repository Layout

- `src/` Rust backtester implementation
- `traders/latest_trader.py` bundled default trader
- `datasets/tutorial/` bundled raw IMC tutorial day CSVs and sample submission log
- `datasets/round1/` bundled raw IMC round 1 CSVs
- `datasets/round2/ ... datasets/round8/` placeholders for future round data
- `runs/` persisted outputs when `--persist` is used
- `runs/<backtest-id>/` combined bundle for persisted multi-day runs, including `combined.log` and `manifest.json`

Licensing

Releases 5

 **v0.3.0** Latest
15 hours ago

[+ 4 releases](#)

Packages

No packages published

Contributors 1

 **GeyzsoN**

Languages

 **Rust** 93.9%  **Makefile** 2.8%  **Shell** 1.8%  **Python** 1.2%  **Dockerfile** 0.3%